Rajalakshmi Engineering College

CYBERSECURITY DEPARTMENT

LABORATORY RECORD MANUAL

Computer Networks (CS23232)

Name of the Student	Tarun.S
Register Number	2116251901172
Class	2nd year (A Section Semester 3)
Academic Year	2026-27
Name of the Lab-In-Charge	Seetha Lakshmi Mam

Reference: CN_LAB_MANUAL_DOCX (1).pdf

EXPERIMENT 1(a)

Basic Networking Commands in Linux

Aim

To practise common Linux terminal commands and understand how they help a user inspect files, processes, memory, and network connectivity.

Introduction

Linux commands are short instructions typed in a terminal. They allow a user to work with files and folders, check running programs, view system resources, and test whether another host can be reached. The exact output depends on the computer, current folder, permissions, and network connection. In this experiment, commands are entered one at a time and the important part of each response is observed.

Commands and their uses

1. ls — Lists the files and directories in the current location. Use `ls -l` for a longer listing and `ls -a` to include hidden entries.

2. echo — Prints text or the value of a variable in the terminal. Example: `echo "Hello Linux"`.

3. ps — Shows a snapshot of processes running at the time the command is executed. `ps` gives a basic list, while `ps aux` gives more detail on many Linux systems.

4. cat — Displays a text file or joins the contents of files. Example: `cat notes.txt`. It is useful for checking a small configuration or text file.

5. free — Displays used and available memory. `free -h` uses easier-to-read units such as MiB or GiB.

6. rm — Removes a file. Example: `rm sample.txt`. The file should be checked before removal because this command may not send it to a recycle bin.

7. top — Shows changing information about running processes, CPU use, and memory use. Press q to quit the display.

8. vmstat — Reports system activity such as memory, processes, and CPU statistics. Example: `vmstat 1 5` requests five updates at one-second intervals.

9. ping — Sends test packets to a destination and reports replies and timing when the destination responds. Example: `ping -c 4 example.com` on many Linux distributions.

Procedure and command practice

Procedure

1. Open the Linux terminal or the terminal provided by the laboratory environment.
2. Run `pwd` to note the current working directory.
3. Execute `ls` and identify the files and folders displayed.
4. Try `echo` with a short message, then use `cat` on an existing text file.
5. Run `ps`, `free -h`, `top`, and `vmstat 1 5` to observe system information.
6. Use `ping -c 4` with an approved host to check connectivity.
7. If file deletion is part of the lab, create a disposable test file first and remove only that file with `rm`.
8. Write down the command, what it displays, and one useful point learned from it.

Sample terminal commands

```
pwd
ls -l
echo "Computer Networks Lab"
ps
cat notes.txt
free -h
vmstat 1 5
ping -c 4 example.com
```

Note: `notes.txt` and the destination host are examples. Replace them with a file and host that are available in the lab environment. Do not run destructive commands on important files.

Observation, learning and result

Observation table

Command	What to note
ls	Names and types of entries displayed
echo	Text printed in the terminal
ps / top	Process names and resource information
cat	Content of the selected text file
free / vmstat	Memory and system activity figures
ping	Replies, packet loss and response time
rm	Whether the test file was removed

Discussion

These commands perform different jobs, but together they provide a basic picture of a Linux system. File commands help manage information, process and memory commands show system status, and ping provides a simple network reachability check. A failed ping does not always prove that a host is down, because a firewall may block ICMP replies.

Result

The basic Linux terminal commands were practised and their purposes were understood by observing the command responses in the laboratory environment.

Viva-Voce Questions

Q. What does ls do?

Answer: It displays files and directories.

Q. Why is ps used?

Answer: It shows information about running processes.

Q. What does free -h display?

Answer: It reports memory usage in readable units.

Q. What is ping used for?

Answer: It checks whether a destination responds to network test packets.

Q. Why should rm be used carefully?

Answer: It deletes files and may not provide an easy recovery option.

EXPERIMENT 1(b)

Basic Networking Commands in Windows

Aim

To use common Windows Command Prompt commands for directory navigation, screen control, system inspection, and network testing.

Introduction

Windows Command Prompt accepts commands that help users move between folders, list files, clear the screen, display system details, and test network connectivity. These commands are useful when diagnosing basic computer and network problems. Some commands have different options depending on the Windows version.

Commands studied

1. cd — Displays or changes the current directory. Use `cd ..` to move to the parent folder; `cd /d D:\Work` can change drive and folder together.

2. cls — Clears the Command Prompt window. It affects only the visible screen, not files or settings.

3. ipconfig — Displays IP configuration such as IPv4 address, subnet mask, and default gateway. `ipconfig /all` shows more adapter details.

4. echo — Prints a message or displays whether command echoing is enabled. Example: `echo Network Lab`.

5. del — Deletes one or more files. It should be tested only on a disposable file created for the experiment.

6. find — Searches text in a file or input. Example: `find "error" log.txt`.

7. systeminfo — Displays detailed operating-system and hardware information, including Windows version and memory details.

8. ping — Sends test packets to a host and displays replies and timing. Example: `ping 127.0.0.1` checks the local TCP/IP stack.

Procedure and sample commands

Procedure

1. Open Command Prompt from the Windows Start menu.
2. Type `cd` and note the current folder. Use `cd ..` to move one level up.
3. Run `cls` and observe that the visible command history is cleared from the window.
4. Run `ipconfig` and identify the active adapter, IPv4 address, subnet mask, and gateway.
5. Use `echo` to print a sample message.
6. Use `systeminfo` to view operating-system details.
7. Run `ping 127.0.0.1` and, if permitted, ping the lab gateway or another approved host.
8. If practising `find` or `del`, create a small text file first and use only that test file.

Sample commands

```
cd
cd ..
cls
ipconfig
ipconfig /all
echo Computer Networks Lab
systeminfo
ping 127.0.0.1
find "network" sample.txt
```

For the delete command, create a temporary file such as `test-delete.txt` and remove only that file after confirming its name.

Observation and result

Observation table

Command	Expected type of information
cd	Current folder or navigation result
cls	Cleared command window
ipconfig	IP address, subnet mask and gateway
echo	Message printed on screen
find	Lines containing the searched text
systeminfo	Windows and computer details
ping	Reply status and round-trip time

Discussion

The commands provide different views of the computer. The ipconfig command is especially useful for checking network settings, while ping gives a quick test of reachability. The loopback address 127.0.0.1 tests the local networking stack and does not test the connection to another computer.

Result

Basic Windows Command Prompt commands were executed or reviewed, and their uses in file navigation, system inspection, and connectivity testing were understood.

Viva-Voce Questions

Q. What is the purpose of ipconfig?

Answer: It displays network configuration for Windows adapters.

Q. What does cls do?

Answer: It clears the visible Command Prompt screen.

Q. What is systeminfo used for?

Answer: It displays detailed operating-system and computer information.

Q. What does 127.0.0.1 mean?

Answer: It is the IPv4 loopback address for the local computer.

Q. What does ping help test?

Answer: It helps check whether a network destination responds.

EXPERIMENT 2

Socket Programming in Python

Aim

To understand client-server communication in Python by creating simple TCP and UDP echo programs.

Introduction

A socket is an endpoint that lets programs exchange data. In a client-server setup, the server waits while the client begins communication. Python provides the `socket` module. TCP offers reliable, ordered delivery through a connection; UDP sends datagrams without guaranteeing delivery or order.

Important socket terms

• AF_INET represents IPv4 addressing.

• SOCK_STREAM selects TCP; SOCK_DGRAM selects UDP.

• bind() assigns a local address and port.

• listen() waits for TCP connections; accept() accepts one.

• send() / recv() exchange data on a connected socket.

Algorithm and TCP program

TCP server algorithm

1. Create a TCP socket.
2. Bind it to localhost and port 55555.
3. Start listening for a client.
4. Accept a connection and read the incoming message.
5. Send the same message back to the client.
6. Close the client connection and stop the server when required.

TCP server code

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 55555))
server.listen(1)
print("Waiting for a client...")

try:
    while True:
        client, address = server.accept()
        with client:
            message = client.recv(1024).decode("utf-8")
            print("Client says:", message)
            client.sendall(message.encode("utf-8"))
        break
finally:
    server.close()
```

TCP client code

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
try:
    client.connect(("127.0.0.1", 55555))
    message = input("Enter a message: ")
    client.sendall(message.encode("utf-8"))
    reply = client.recv(1024).decode("utf-8")
    print("Reply from server:", reply)
finally:
    client.close()
```

Run the server first, then run the client in a second terminal. Both programs use localhost, so the test stays on the same computer.

UDP program and comparison

UDP server code

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(("127.0.0.1", 8000))
print("UDP server is ready")
try:
    data, address = server.recvfrom(1024)
    print("Client says:", data.decode("utf-8"))
    server.sendto(data, address)
finally:
    server.close()
```

UDP client code

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
try:
    message = input("Enter a message: ")
    client.sendto(message.encode("utf-8"), ("127.0.0.1", 8000))
    data, address = client.recvfrom(1024)
    print("Reply from server:", data.decode("utf-8"))
finally:
    client.close()
```

TCP and UDP comparison

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable, ordered byte stream	No delivery/order guarantee
Typical use	Web sessions, file transfer	Streaming, games, simple queries
Python type	SOCK_STREAM	SOCK_DGRAM

Observations, result and viva

Observations

When the client sends a message, the server receives the bytes, decodes them into text, and sends the same text back. The client then displays the reply. TCP requires the server to listen and accept a connection; the UDP server instead waits for a datagram. In both examples, the programs are limited to one message for clarity.

Expected output example

```
Server: Waiting for a client...
Client says: Hello

Client: Enter a message: Hello
Reply from server: Hello
```

Result

Basic client-server communication was demonstrated using Python sockets. TCP and UDP communication methods and their main differences were understood.

Viva-Voce Questions

Viva-Voce Questions

Q. What is a socket?

Answer: It is an endpoint used by programs to send and receive data.

Q. What does TCP provide?

Answer: It provides reliable, ordered delivery through a connection.

Q. Why is UDP called connectionless?

Answer: It sends datagrams without setting up a connection first.

Q. What is the purpose of bind()?

Answer: It assigns a local address and port to a socket.

Q. Why encode a message before sending it?

Answer: Socket data is sent as bytes, so text must be converted to bytes.

EXPERIMENT 4

Packet Sniffing Using Raw Sockets in Python

Aim

To study packet capture concepts and inspect network packet information in an authorized laboratory environment.

Introduction

Packet sniffing means capturing network packets and examining their headers and contents. Network administrators use packet analysis to troubleshoot connectivity, understand protocols, and investigate suspicious activity. A raw socket may provide low-level access to packets, but operating systems restrict this access because packet data can contain private information. This experiment should be performed only on a computer and network for which permission has been granted.

Basic concepts

• A packet contains control information in headers and may also contain application data.

• A MAC address identifies a network interface at the data-link layer.

• An IP address identifies a network interface at the network layer.

• A protocol field helps identify what type of packet or payload is present.

• Promiscuous capture can expose traffic beyond packets addressed to the local interface, depending on the network and capture setup.

Algorithm and code outline

Algorithm

1. Determine the local host address and create a raw socket if the operating system supports it.
2. Configure the socket according to the operating system’s requirements and permissions.
3. Receive packet bytes from the local interface.
4. Read the relevant header fields using byte offsets and structure formats.
5. Convert address bytes into a readable format.
6. Display only the information needed for troubleshooting and stop capture when the task is complete.

Illustrative header parsing code

```
import struct

def format_mac(raw):
    return ":".join(f"{byte:02X}" for byte in raw)

def parse_ethernet_header(frame):
    # Ethernet header is 14 bytes: destination, source, type
    if len(frame) < 14:
        raise ValueError("Frame is shorter than an Ethernet header")
    destination, source, protocol = struct.unpack("!6s6sH", frame[:14])
    return {
        "destination_mac": format_mac(destination),
        "source_mac": format_mac(source),
        "ether_type": f"0x{protocol:04X}",
        "payload_length": len(frame) - 14,
    }

# Example use: pass a captured Ethernet frame as bytes.
# Details of live capture differ by operating system.
```

This short example explains how a frame header can be parsed. Live raw-socket capture is operating-system dependent; a typical Ethernet-frame parser cannot be assumed to work unchanged on every platform.

How to interpret packet information

Reading the fields

Destination MAC: The link-layer interface to which the frame is addressed.

Source MAC: The link-layer interface that sent the frame.

EtherType: A value that commonly indicates the payload type, such as IPv4 or ARP.

Payload length: The number of bytes after the header used in this simplified example.

Capture timestamp: When the packet was observed, if the capture tool records it.

Practical procedure

1. Use a permitted test machine and confirm that packet capture is allowed.
2. Prefer Wireshark or another approved capture tool for a portable laboratory exercise.
3. Select the correct interface and capture a short sample of ordinary test traffic.
4. Choose a packet and inspect its Ethernet and IP headers.
5. Record source and destination addresses and identify the protocol.
6. Stop capture and close the file after recording observations.

Precautions

Do not capture traffic from other users or networks without authorization. Do not attempt to collect passwords, session tokens, or private message content. Run capture tools with elevated permissions only when necessary and only in the approved lab.

Observation, result and viva

Observation table

Field	Observation to record
Source MAC	Address shown by the capture tool
Destination MAC	Address shown by the capture tool
Protocol / EtherType	ARP, IPv4, IPv6, or other identified type
Packet length	Length reported for the selected packet
Notes	Purpose of the traffic and any notable header fields

Result

The principles of packet sniffing and Ethernet-header interpretation were studied. Packet information can be used for network troubleshooting when capture is performed with proper authorization.

Viva-Voce Questions

Viva-Voce Questions

Q. What is packet sniffing?

Answer: It is capturing and examining packets travelling through a network.

Q. What is a raw socket?

Answer: It is a socket interface that can provide low-level access to network packets.

Q. Why may elevated permission be needed?

Answer: Packet capture accesses low-level network information restricted by the operating system.

Q. What is a MAC address?

Answer: It is a link-layer address used to identify an interface on a local network.

Q. Why must packet capture be authorized?

Answer: Captured traffic can include sensitive information belonging to other users.

EXPERIMENT 6

Nmap Live Host Discovery

Aim

To learn how Nmap can identify reachable hosts in an authorized test network using different host-discovery methods.

Introduction

Nmap (Network Mapper) is a network exploration tool used to identify hosts and learn which ports or services may be available. Live-host discovery is an early step in mapping a network. Different discovery methods work in different environments: ARP is useful on a local Ethernet segment, ICMP may be filtered, and TCP or UDP probes may help when ICMP responses are blocked. Scanning must be limited to a lab or network for which permission has been granted.

Discovery methods

• ARP discovery: asks which local devices have a given IPv4 address on the local network.

• ICMP discovery: uses ICMP echo or other supported probes to check whether a host responds.

• TCP probe: tests a selected TCP port or sends a TCP discovery probe.

• UDP probe: sends a UDP probe and interprets any response, while silence can be ambiguous.

• Reverse DNS: attempts to find a name associated with an IP address.

Procedure and example commands

Procedure

1. Open the approved Nmap lab or TryHackMe room and read the scope before scanning.
2. Identify the target IP address or subnet supplied for the exercise.
3. Start with a discovery-only scan to see which hosts respond.
4. If permitted, compare ARP, ICMP, and TCP/UDP discovery behaviour.
5. Record the command used, target range, and hosts reported as up.
6. Use reverse-DNS lookup where the lab asks for host names.
7. Compare the results and note that a non-response is not conclusive proof that a host is offline.

Example commands (authorized targets only)

```
nmap -sn 192.168.1.0/24
nmap -sn -PE 192.168.1.0/24
nmap -sn -PS80,443 192.168.1.10
nmap -sn -PU53 192.168.1.10
nmap -sn -R 192.168.1.10
```

The addresses above are examples for a private lab subnet. Replace them with the exact target provided by your instructor. The options have slightly different effects depending on privileges, operating system, and network layout.

Observations and interpretation

Observation table

Method	What to observe	Possible limitation
ARP	Local IP-to-MAC discovery	Generally applies to the local layer-2 network
ICMP	Echo or other ICMP responses	May be blocked by host or network firewall
TCP probe	Whether the target responds to selected TCP probe	Result depends on port and filtering
UDP probe	Any UDP or related response	No response may be inconclusive
Reverse DNS	Name returned for an IP address	Many addresses have no reverse name

Discussion

Host discovery results depend on network configuration and security controls. A host may be active but appear silent because probes are filtered. Likewise, an address can respond through a gateway or network device rather than the expected endpoint. The scope and the exact command should therefore be written down so results can be interpreted correctly.

Result

Nmap live-host discovery methods were studied in the approved lab environment, and the differences between ARP, ICMP, TCP/UDP probes, and reverse-DNS lookup were understood.

Viva-Voce Questions

Q. What is Nmap?

Answer: A network discovery and security-auditing tool.

Q. What is host discovery?

Answer: Checking which hosts in a target range appear reachable.

Q. Why might ping fail for an active host?

Answer: A firewall or network policy may block ICMP replies.

Q. What is ARP mainly used for?

Answer: Mapping an IPv4 address to a link-layer address on a local network.

Q. Why should scans have an approved scope?

Answer: Scanning systems without permission may violate policy or law.

EXPERIMENT 7

Anonymous FTP Access Check Using Python

Aim

To understand FTP connections and test anonymous login against a specifically authorized lab FTP server.

Introduction

FTP is a protocol used to transfer files between a client and a server. Some servers allow anonymous access, often with the username `anonymous`. This can be appropriate for public file repositories, but it may be a security concern if private files are exposed. A scanner should only test a server that the student owns or has explicit permission to assess.

Python module

Python's `ftplib` module provides a client interface for FTP. The program connects to a host, attempts the configured login, reports whether access is accepted, and closes the connection. A timeout prevents the script from waiting indefinitely for a slow host.

Algorithm and program

Algorithm

1. Read the lab FTP server address from the instructor or lab configuration.
2. Create an FTP client and set a reasonable connection timeout.
3. Connect to the server on the configured FTP control port.
4. Attempt anonymous login with a generic anonymous identity.
5. Report whether the server accepted or rejected the login.
6. Close the FTP connection even if an error occurs.

Program

```
from ftplib import FTP, all_errors

def check_anonymous_ftp(host):
    ftp = FTP()
    try:
        ftp.connect(host=host, port=21, timeout=5)
        reply = ftp.login(user="anonymous", passwd="student@example.invalid")
        print(f"{host}: anonymous login accepted ({reply})")
        return True
    except all_errors as error:
        print(f"{host}: login failed or FTP unavailable: {error}")
        return False
    finally:
        try:
            ftp.quit()
        except all_errors:
            ftp.close()

# Use only the host assigned for your authorized lab.
# check_anonymous_ftp("192.0.2.10")
```

The sample address is reserved for documentation and is not a real target. Replace it with your instructor-provided lab host. The script intentionally checks one host instead of sweeping unrelated addresses.

Testing and security interpretation

Procedure

1. Confirm the target hostname or IP address and obtain permission to test it.
2. Check that the lab FTP service is available and that the network permits the connection.
3. Run the program from a Python environment with the standard library installed.
4. Record whether the connection succeeds and whether anonymous login is accepted.
5. If access is accepted, do not browse or download files unless the exercise explicitly authorizes that action.
6. Close the program and report the finding responsibly to the instructor.

How to interpret the result

“Anonymous login accepted” means the server allowed the login attempt. It does not automatically mean that the server is misconfigured: some public FTP services are designed for anonymous downloads. The risk depends on what files and operations the account can access. “Login failed” may mean anonymous access is disabled, but it can also indicate a network failure, incorrect target, service outage, or timeout.

Precautions

Do not scan large address ranges or test public hosts without permission. Avoid using real passwords in scripts or logs. If a server permits anonymous access, examine only the content allowed by the lab instructions.

Observation, result and viva

Observation table

Item	Observation
Target host	Instructor-provided lab address
Connection	Successful / timed out / refused
Anonymous login	Accepted / rejected / not determined
Error message	Record a short relevant message
Conclusion	Public anonymous service or possible access-control issue

Result

A Python FTP access check was prepared to demonstrate how anonymous login can be tested against an approved lab server. The result should be recorded from the actual run rather than assumed in advance.

Viva-Voce Questions

Viva-Voce Questions

Q. What is FTP?

Answer: A protocol used to transfer files between a client and server.

Q. What is anonymous FTP?

Answer: A login mode that allows access without an individual user account, where enabled.

Q. Which Python module is used here?

Answer: The ftplib module.

Q. Does accepted anonymous login always mean a vulnerability?

Answer: No. Some servers intentionally provide public files; the permissions and exposed content matter.

Q. Why is a timeout useful?

Answer: It prevents the program from waiting indefinitely for a response.

EXPERIMENT 9

Design a Simple Network Topology Using Cisco Packet Tracer

Aim

To create and test a small routed network containing one router, two switches, and four PCs.

Introduction

Cisco Packet Tracer is a network simulation tool that allows devices and links to be assembled virtually. In this experiment, each switch connects two PCs. A router connects the two separate IPv4 networks so that devices on one side can communicate with devices on the other side. Each PC needs an IP address, subnet mask, and default gateway that match its network.

Devices required

• One router with two available Ethernet interfaces.

• Two access switches.

• Four PCs.

• Copper Ethernet connections between each PC and switch and between switches and router.

Topology and IP addressing

Logical layout

```
PC1 ---- Switch1 ---- Router ---- Switch2 ---- PC3
PC2 ----/       G0/0/0  G0/0/1       \---- PC4
```

Addressing plan

Device / interface	IPv4 address	Subnet mask	Gateway
Router G0/0/0	192.168.1.1	255.255.255.0	—
PC1	192.168.1.2	255.255.255.0	192.168.1.1
PC2	192.168.1.3	255.255.255.0	192.168.1.1
Router G0/0/1	192.168.2.1	255.255.255.0	—
PC3	192.168.2.2	255.255.255.0	192.168.2.1
PC4	192.168.2.3	255.255.255.0	192.168.2.1

Procedure

1. Place the router, two switches, and four PCs in the workspace and rename them.
2. Connect PC1 and PC2 to Switch1; connect PC3 and PC4 to Switch2.
3. Connect Switch1 to the router’s first interface and Switch2 to its second interface.
4. Open each PC’s IP Configuration window and enter the address, mask, and gateway from the table.
5. Open the router CLI and configure both router interfaces.

Router configuration and testing

Example router CLI configuration

```
enable
configure terminal
interface gigabitEthernet 0/0/0
 ip address 192.168.1.1 255.255.255.0
 no shutdown
exit
interface gigabitEthernet 0/0/1
 ip address 192.168.2.1 255.255.255.0
 no shutdown
exit
end
show ip interface brief
```

Interface names vary by router model in Packet Tracer. If the device uses different interface names, select the actual ports shown in the router.

Connectivity test

1. From PC1, open Desktop → Command Prompt and run `ping 192.168.1.1`.

2. From PC1, ping PC2 at `192.168.1.3`.

3. From PC1, ping PC3 at `192.168.2.2` to test routing across the two networks.

4. Repeat from a PC on the second network if required.

5. If a ping fails, check cabling, interface status, address/mask, and gateway settings before changing other configuration.

Troubleshooting notes

The router interfaces must be enabled, and each PC must use a gateway on its own subnet. An address duplicated on two devices can cause unpredictable communication. The first ping may take longer while address resolution completes. Record actual replies or errors instead of writing that every test succeeded when it did not.

Observation, result and viva

Observation table

Test	Purpose	Actual observation
PC1 → 192.168.1.1	Reach first router interface	Fill after test
PC1 → 192.168.1.3	Test same subnet	Fill after test
PC1 → 192.168.2.2	Test inter-network routing	Fill after test
PC3 → 192.168.2.1	Reach second router interface	Fill after test

Result

A two-subnet topology was designed in Cisco Packet Tracer. IP addressing and router configuration were used to test communication within and between the networks; the final result should reflect the ping tests performed.

Viva-Voce Questions

Viva-Voce Questions

Q. What does a switch do?

Answer: It connects devices within a local network and forwards frames using MAC addresses.

Q. What does a router do?

Answer: It forwards packets between different IP networks.

Q. Why is a default gateway needed?

Answer: It tells a host where to send traffic for other networks.

Q. What does /24 mean in IPv4?

Answer: It corresponds to the subnet mask 255.255.255.0.

Q. What does no shutdown do on a router interface?

Answer: It administratively enables the interface.

EXPERIMENT 10

Customize a Switch with Network Modules in Cisco Packet Tracer

Aim

To explore the physical layout of a switch, add a supported network module, and practise basic switch configuration.

Introduction

A switch connects devices in a local area network. Depending on the model, a switch may have fixed ports or expansion slots that accept supported modules. A module can provide extra interfaces or a different connection type. Packet Tracer models do not all support the same modules, so the available hardware depends on the selected device.

Procedure: hardware module

1. Open Cisco Packet Tracer and add a switch model that supports expansion modules.
2. Select the switch and open the Physical tab.
3. If the switch requires it, turn its power off before changing modules.
4. Choose a compatible module from the available list and place it in an empty slot.
5. Turn the switch on and check whether the new interface or module is visible.
6. Save the Packet Tracer activity.

Basic switch configuration

Example CLI commands

```
enable
configure terminal
hostname LabSwitch
banner motd #Authorized laboratory use only#
enable secret class123
line console 0
 password console123
 login
exit
end
show vlan brief
```

The passwords above are examples for a simulated classroom activity only. Use the credentials and password policy provided by your instructor, and do not reuse laboratory passwords for real systems.

VLAN concepts

A VLAN separates a switch into logical broadcast domains. Naming a VLAN helps identify its purpose, and assigning an access port to that VLAN places the connected device in the selected logical network. The exact interface names and available commands depend on the switch model.

VLAN example, observations and result

Example VLAN commands

```
enable
configure terminal
vlan 10
 name STUDENTS
exit
interface fastEthernet 0/1
 switchport mode access
 switchport access vlan 10
exit
end
show vlan brief
```

These commands are an illustrative configuration for a switch model that supports them. A switch virtual interface IP address may be configured for management, but an ordinary Layer 2 switch does not route between VLANs by itself.

Observation table

Step	What to verify
Module installation	Module appears in the physical layout after power-on
Hostname / banner	Configured name and banner are shown where applicable
VLAN creation	VLAN 10 appears in show vlan brief
Port assignment	Selected access port is listed under VLAN 10
Saved activity	Topology and configuration remain after saving

Result

The physical switch layout and module options were explored in Packet Tracer. Basic switch identification, access settings, and VLAN configuration were practised where supported by the selected model.

Viva-Voce Questions

Viva-Voce Questions

Q. What is a network module?

Answer: A component that adds supported interfaces or capabilities to a device.

Q. Why must a compatible module be selected?

Answer: A switch accepts only modules supported by its hardware model.

Q. What is a VLAN?

Answer: A logical division of a switched network.

Q. What does hostname configure?

Answer: The name used to identify the device in its CLI.

Q. Does a Layer 2 switch route between VLANs by itself?

Answer: Normally no; inter-VLAN routing requires a router or a Layer 3 switch.

EXPERIMENT 12

Capture and Analyze Network Traffic Using Wireshark

Aim

To capture a short sample of network traffic, save it, and inspect packets from common protocols using Wireshark.

Introduction

Wireshark is a graphical protocol analyzer. It records packets visible to the selected network interface and displays their protocol layers. It can help explain how devices resolve addresses, exchange data, and use protocols such as TCP, UDP, DNS, DHCP, ARP, and ICMP. Captures can contain sensitive information, so only authorized traffic should be collected.

Preparation

1. Install Wireshark from its official source and install the required capture driver if prompted.

2. Open Wireshark and identify the active Wi-Fi or Ethernet interface.
3. Confirm that capturing on this interface is allowed in the lab.
4. Start a short capture and generate ordinary test traffic, such as opening an approved website or pinging a lab host.

Capture filters and display filters

Capture filters

A capture filter limits which packets are collected. It is set before or during capture, depending on the interface. Examples include:

```
tcp
udp
arp
icmp
port 53
udp port 67 or udp port 68
```

Display filters

A display filter hides packets that do not match the expression while leaving the captured data intact. Common examples are:

```
tcp
udp
ip
http
arp
icmp
dns
bootp
```

A filter expression must match Wireshark's syntax. DHCP traffic is commonly identified through the BOOTP/DHCP protocol fields; the exact display may depend on Wireshark version and packet type.

Saving a capture

Stop the capture before saving when the required sample has been collected. Choose File → Save As and use a descriptive filename with a .pcapng or .pcap extension. Save only the traffic needed for the experiment and follow the laboratory's rules for retaining or sharing capture files.

Analyzing packets

Packet details

Select a packet in the packet list. The details pane shows the protocol layers and fields, while the bytes pane shows the raw packet representation. Expand the Ethernet, IP, TCP/UDP, and application-protocol sections to learn which fields are present.

Useful analysis features

Follow Stream: Shows a conversation associated with a TCP connection or UDP flow when supported by the selected packet.

Protocol Hierarchy: Summarizes the protocols represented in the capture.

Conversations: Lists communication pairs and traffic statistics.

Endpoints: Lists network endpoints, such as IP and MAC addresses, seen in the capture.

I/O Graphs: Plots packet or byte activity over time to help identify bursts or gaps.

Expert Information: Highlights selected protocol warnings, errors, or retransmission-related observations; these require interpretation rather than automatic assumptions.

Procedure

1. Start capture on the chosen interface.
2. Generate a small amount of approved test traffic.
3. Stop the capture and save it in a suitable file format.
4. Apply a display filter such as `dns` or `icmp`.
5. Select a packet and record its source, destination, protocol, and relevant fields.
6. Open Statistics → Protocol Hierarchy or Conversations and note the useful summary information.

Observation, result and viva

Observation table

Protocol / feature	What to record
ARP	Address-resolution request or reply, if present
IP	Source and destination IP addresses
TCP / UDP	Transport protocol and port information
DNS	Query name and response information, if visible
ICMP	Echo request/reply or other ICMP message
Statistics	Protocols or conversations observed

Result

Network traffic capture, saving, filtering, and packet inspection were studied using Wireshark. The protocols actually present in the saved capture should be listed in the observation table.

Viva-Voce Questions

Viva-Voce Questions

Q. What is Wireshark?

Answer: A tool for capturing and analyzing network packets.

Q. What is the difference between a capture filter and display filter?

Answer: A capture filter limits what is collected; a display filter limits what is shown.

Q. What does DNS do?

Answer: It helps resolve domain names to IP addresses.

Q. What is ARP used for?

Answer: It helps resolve an IPv4 address to a link-layer address on a local network.

Q. Why should packet captures be handled carefully?

Answer: They may contain sensitive information about network activity.